

[NAME OF THE SCHOOL]

[City, State]

COMPUTER SCIENCE (083)

PROJECT REPORT

Session 2026–27

CARBON DATA LOGGER

Personal Carbon Footprint Tracking System

(Python and MySQL)

Submitted by:

[Student Name]

Class XII – [Section]

Roll No.: [Board Roll No.]

Under the guidance of:

[Teacher's Name]

PGT (Computer Science)

CERTIFICATE

This is to certify that **[Student Name]** of Class XII – [Section], **[Name of the School]**, has successfully completed the Computer Science (083) project titled “**Carbon Data Logger – Personal Carbon Footprint Tracking System**” under my guidance during the academic session 2026–27, as prescribed by the Central Board of Secondary Education (CBSE).

The project is the original work of the student and has been carried out with sincerity and dedication.

Internal Examiner

External Examiner

Principal

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my Computer Science teacher, [Teacher's Name], for the valuable guidance, support and encouragement throughout this project.

I am also thankful to our Principal, [Principal's Name], for providing the facilities and an environment that made this work possible.

I thank my parents and friends for their constant support and for helping me test the program and collect sample data.

[Student Name]

Class XII – [Section]

INDEX

S. No.	Topic
1	Introduction
2	Objectives of the Project
3	Features of the Project
4	Hardware and Software Requirements
5	Python Modules and Concepts Used
6	Database Design
7	Program Design and Functions
8	Source Code
9	Output Screenshots
10	Testing
11	Limitations and Future Scope
12	Bibliography

1. Introduction

Every day we do things that release carbon dioxide (CO₂) into the air. Electricity from coal-based power plants, petrol and diesel vehicles, cooking gas and air travel all add to our personal carbon footprint. Most people do not know how much CO₂ their daily activities produce, because there is no simple way to keep track.

The Carbon Data Logger is a menu-driven terminal program written in Python with a MySQL database. It lets one person record everyday activities such as electricity use, travel by car, bus or train, LPG use and flights. For each activity the program estimates the CO₂ released, using the formula:

$$\text{CO}_2 \text{ emission (kg)} = \text{Quantity} \times \text{Emission factor}$$

The emission factor is the amount of CO₂ released by one unit of the activity, for example 0.17 kg of CO₂ for every kilometre travelled by car. The factors are stored in the database, so they can be changed without changing the program.

All records are stored in two related MySQL tables. The user can search, update and delete records, see overall, category-wise and monthly summaries, find the category with the highest emissions, get simple suggestions to reduce emissions, and export the data to a CSV file and a text report.

Note: *The project is educational. The emission factors are illustrative values chosen for this project and are not official scientific data.*

2. Objectives of the Project

- To create awareness about the carbon footprint of everyday activities.
- To build a working Python–MySQL application that stores and processes real-life data.
- To apply Class XII concepts: user-defined functions, exception handling, text and CSV file handling, SQL and Python–SQL connectivity.
- To demonstrate a relational database with a primary key, a foreign key and a join between two tables.
- To use SQL aggregate functions and GROUP BY to produce meaningful summaries.
- To validate user input so that wrong data does not crash the program or enter the database.

3. Features of the Project

The program is controlled from a main menu with 13 options:

Option	Feature	Description
1	Add Carbon Record	Choose a category, enter the date, quantity and an optional note; the program shows the calculation and saves the record after confirmation.
2	View All Records	Shows every record in a neat table, sorted by date.
3	Search Records	Search by record ID, by category name (partial match), by a single date, or between two dates.
4	Update Record	Change the date, category, quantity or note; the CO2 value is recalculated.
5	Delete Record	Shows the record and deletes it only after the user confirms with Y.
6	Carbon Footprint Summary	Total records, total, average, lowest and highest CO2, and the date range.
7	Category-wise Summary	Number of records, total CO2 and percentage share of each category.
8	Monthly Summary	Lists months that have data, then shows the summary of the chosen month.
9	Highest Emission Category	Shows the highest and lowest categories and rule-based suggestions.
10	Export Records to CSV	Writes all records to carbon_records.csv.
11	Export Summary to Text File	Writes a full summary report to carbon_summary.txt.
12	Manage/View Emission Factors	View factors, change a factor, or add a new category.
13	Help	Shows short instructions for using the program.
0	Exit	Closes the database connection and ends the program.

4. Hardware and Software Requirements

4.1 Hardware

Component	Minimum requirement
Processor	Intel Core i3 / AMD equivalent or better
RAM	4 GB
Hard disk	500 MB free space
Input / Output	Keyboard, monitor

4.2 Software

Software	Details
Operating system	Windows 10/11, Linux or macOS
Programming language	Python 3.8 or later
Database	MySQL Server 8.0 (or MariaDB)
Python package	mysql-connector-python
Editor	IDLE / Visual Studio Code

4.3 How to run the project

1. Install Python 3 and MySQL Server.
2. Install the connector: `pip install mysql-connector-python`
3. In `carbon_logger.py`, set `DB_PASSWORD` to your MySQL password (it is "root" by default).
4. Optional: run `SOURCE database_setup.sql`; once in the MySQL client to load sample data.
5. Run the program: `python carbon_logger.py`

5. Python Modules and Concepts Used

5.1 Modules

Module	Use in the project
<code>mysql.connector</code>	Connects Python to MySQL and runs SQL queries (<code>connect()</code> , <code>cursor()</code> , <code>execute()</code> , <code>commit()</code> , <code>fetchone()</code> , <code>fetchall()</code> , <code>rowcount</code>).
<code>csv</code>	Exports records to a CSV file using <code>writer()</code> , <code>writerow()</code> and <code>writerows()</code> .
<code>datetime</code>	Checks that dates are valid (<code>strptime</code>) and gives today's date.

5.2 Syllabus concepts demonstrated

Concept	Where it is used
User-defined functions	The whole program is divided into 42 small functions, called from <code>main()</code> .
Parameters and return values	<code>calculate_emission(quantity, emission_factor)</code> , <code>validate_date(date_text)</code> , <code>get_grand_total(rows)</code> .
Global and local scope	<code>connect_database()</code> uses the global keyword for connection and cursor.
Lists, tuples, dictionaries	<code>DEFAULT_CATEGORIES</code> (list of tuples), <code>MONTH_NAMES</code> (tuple), <code>SUGGESTIONS</code> and <code>menu_actions</code> (dictionaries).
Exception handling	<code>try-except-finally</code> for invalid numbers, dates, MySQL errors, duplicate names, file errors and Ctrl+C.
Text file handling	<code>export_summary_to_text()</code> uses <code>open()</code> , <code>write()</code> and <code>close()</code> in a finally block.
CSV file handling	<code>export_to_csv()</code> uses a with block and <code>csv.writer</code> .
SQL DDL	<code>CREATE DATABASE</code> , <code>CREATE TABLE</code> with <code>PRIMARY KEY</code> , <code>FOREIGN KEY</code> , <code>NOT NULL</code> and <code>UNIQUE</code> .
SQL DML	<code>INSERT</code> , <code>SELECT</code> , <code>UPDATE</code> , <code>DELETE</code> .
SQL clauses and operators	<code>WHERE</code> , <code>BETWEEN</code> , <code>LIKE</code> , <code>DISTINCT</code> , <code>ORDER BY</code> , aliases.
Aggregate functions	<code>COUNT()</code> , <code>SUM()</code> , <code>AVG()</code> , <code>MIN()</code> , <code>MAX()</code> with <code>GROUP BY</code> .
Joins	<code>INNER JOIN</code> (equi-join) between <code>emission_records</code> and <code>categories</code> .
Python–SQL connectivity	Parameterised queries with the <code>%s</code> format specifier.

6. Database Design

Database name: carbon_logger. It has two related tables.

6.1 Table: categories

Stores each type of activity with its unit and emission factor.

Field	Data type	Constraint	Description
category_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID of the category
category_name	VARCHAR(30)	NOT NULL, UNIQUE	Name, e.g. Car Travel
unit	VARCHAR(10)	NOT NULL	Unit, e.g. km, kWh, kg
emission_factor	FLOAT	NOT NULL	kg CO2 per unit

6.2 Table: emission_records

Stores one row for every activity recorded by the user.

Field	Data type	Constraint	Description
record_id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique ID of the record
record_date	DATE	NOT NULL	Date of the activity
category_id	INT	NOT NULL, FOREIGN KEY	Refers to categories(category_id)
quantity	FLOAT	NOT NULL	Amount, in the category's unit
co2_emission	FLOAT	NOT NULL	Calculated CO2 in kg
note	VARCHAR(100)	–	Optional note

6.3 Relationship between the tables

One category can have many records, and each record belongs to exactly one category (a one-to-many relationship). The column category_id in emission_records is a foreign key that refers to the primary key of categories. Because of this:

- a record cannot be saved with a category that does not exist;
- a category that still has records cannot be deleted;
- the category name and unit are stored only once and are fetched with an INNER JOIN whenever records are displayed.

6.4 Default data in the categories table

Degree of the table = 4, cardinality = 6 (before the user adds any category).

category_id	category_name	unit	emission_factor
1	Electricity	kWh	0.82
2	Car Travel	km	0.17

category_id	category_name	unit	emission_factor
3	Bus Travel	km	0.10
4	Train Travel	km	0.04
5	LPG	kg	2.98
6	Flight	km	0.15

The emission factors are illustrative values for this project.

7. Program Design and Functions

7.1 Flow of the program

1. The program starts by calling `main()`.
2. `main()` connects to MySQL, creates the database and tables if they do not exist, and adds the default categories if the table is empty. If the connection fails, a clear message is shown and the program stops.
3. `main()` shows the main menu in a loop and reads the user's choice.
4. The dictionary `menu_actions` links each choice to its function, which is then called.
5. Database errors inside an option are caught, so the program returns to the menu instead of stopping.
6. Choosing 0 ends the loop; the finally block always closes the cursor and the connection.

7.2 List of functions

Function	Purpose
<code>connect_database()</code>	Connects to MySQL and creates/selects the database.
<code>create_tables()</code>	Creates the two tables if they do not exist.
<code>insert_default_categories()</code>	Inserts the six default categories into an empty table.
<code>get_integer(prompt)</code>	Repeats the question until a whole number is entered.
<code>get_positive_number(prompt)</code>	Repeats the question until a number greater than zero is entered.
<code>validate_date(date_text)</code>	Returns the date in YYYY-MM-DD form, or None if it is invalid.
<code>get_date(prompt)</code>	Asks for a valid date; Enter gives today's date.
<code>display_categories()</code>	Shows all categories with their units and factors.
<code>get_category(category_id)</code>	Fetches one category from MySQL.
<code>choose_category()</code>	Lets the user pick a valid category.
<code>calculate_emission(q, f)</code>	Returns quantity $\times$ emission factor, rounded to 2 places.
<code>add_record()</code>	Adds a new record after showing the calculation.
<code>print_records(records)</code>	Prints records as a formatted table.
<code>view_all_records()</code>	Displays all records using an INNER JOIN.
<code>get_record(record_id)</code>	Fetches one record by its ID.
<code>search_menu()</code> and <code>search_by_...()</code>	Search by ID, category (LIKE), date, or date range (BETWEEN).
<code>update_record()</code>	Changes a record and recalculates its CO2.
<code>delete_record()</code>	Deletes a record after confirmation.
<code>get_overall_totals()</code>	Runs COUNT, SUM, AVG, MIN and MAX on all records.
<code>get_category_totals()</code>	Totals per category with GROUP BY, highest first.
<code>get_grand_total(rows)</code>	Adds the totals of all categories.
<code>overall_summary()</code>	Displays the overall summary.
<code>category_summary()</code>	Displays totals and percentage share by category.

Function	Purpose
<code>show_available_months()</code>	Lists months with records using DISTINCT.
<code>monthly_summary()</code>	Summary for one chosen month.
<code>generate_suggestions(rows)</code>	Rule-based tips for categories with 20% or more of total CO2.
<code>highest_emission_category()</code>	Shows the highest and lowest categories and suggestions.
<code>export_to_csv()</code>	Writes all records to <code>carbon_records.csv</code> .
<code>export_summary_to_text()</code>	Writes a summary report to <code>carbon_summary.txt</code> .
<code>update_emission_factor()</code>	Changes the factor of a category.
<code>add_category()</code>	Adds a new category with checks for length and duplicates.
<code>manage_emission_factors()</code>	Sub-menu for viewing and managing factors.
<code>show_help()</code>	Displays help text.
<code>display_main_menu()</code>	Prints the main menu.
<code>main()</code>	Starting point; runs the menu loop.

8. Source Code

8.1 database_setup.sql

```
-- =====
-- Carbon Data Logger - Database Setup Script
-- Creates the database, both tables and the default categories.
-- Run ONCE in the MySQL client: SOURCE database_setup.sql;
-- (carbon_logger.py also creates these automatically if missing.)
-- =====

CREATE DATABASE IF NOT EXISTS carbon_logger;
USE carbon_logger;

-- Table 1: activity categories with their unit and emission factor
CREATE TABLE IF NOT EXISTS categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(30) NOT NULL UNIQUE,
    unit VARCHAR(10) NOT NULL,
    emission_factor FLOAT NOT NULL
);

-- Table 2: one row for every activity recorded by the user
CREATE TABLE IF NOT EXISTS emission_records (
    record_id INT AUTO_INCREMENT PRIMARY KEY,
    record_date DATE NOT NULL,
    category_id INT NOT NULL,
    quantity FLOAT NOT NULL,
    co2_emission FLOAT NOT NULL,
    note VARCHAR(100),
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
);

-- Default categories.
-- NOTE: emission factors (kg CO2 per unit) are ILLUSTRATIVE project
-- values for learning purposes only. They can be changed from menu 12.
INSERT INTO categories (category_name, unit, emission_factor) VALUES
    ('Electricity', 'kWh', 0.820),
    ('Car Travel', 'km', 0.170),
    ('Bus Travel', 'km', 0.100),
    ('Train Travel', 'km', 0.040),
    ('LPG', 'kg', 2.980),
    ('Flight', 'km', 0.150);

-- Some sample records for testing (optional)
INSERT INTO emission_records (record_date, category_id, quantity, co2_emission, note) VALUES
    ('2026-08-02', 1, 12.50, 10.25, 'Daily home use'),
    ('2026-08-03', 2, 20.00, 3.40, 'Trip to market'),
    ('2026-08-05', 3, 15.00, 1.50, 'School bus'),
    ('2026-09-01', 5, 2.00, 5.96, 'Cooking gas'),
    ('2026-09-10', 6, 800.00, 120.00, 'Flight to Mumbai');

-- Useful queries for checking the data
SELECT * FROM categories;

SELECT r.record_id, r.record_date, c.category_name, r.quantity, c.unit, r.co2_emission
FROM emission_records r INNER JOIN categories c ON r.category_id = c.category_id
ORDER BY r.record_date;

SELECT c.category_name, COUNT(*), SUM(r.co2_emission)
FROM emission_records r INNER JOIN categories c ON r.category_id = c.category_id
GROUP BY c.category_name;
```


8.2 carbon_logger.py

```
# =====
# CARBON DATA LOGGER - Personal Carbon Footprint Tracking System
# CBSE Class XII Computer Science Project (Python + MySQL)
#
# Records everyday activities (electricity, travel, LPG, flights),
# estimates their CO2 emissions and stores them in MySQL.
# Emission factors are ILLUSTRATIVE project values, not official data.
# =====

import csv
import datetime
import mysql.connector

# MySQL connection settings - change the password to match your system
DB_HOST = "localhost"
DB_USER = "root"
DB_PASSWORD = "root"
DB_NAME = "carbon_logger"

# Output file names
CSV_FILE = "carbon_records.csv"
SUMMARY_FILE = "carbon_summary.txt"

# Default categories: (name, unit, illustrative kg CO2 per unit)
DEFAULT_CATEGORIES = [
    ("Electricity", "kWh", 0.82),
    ("Car Travel", "km", 0.17),
    ("Bus Travel", "km", 0.10),
    ("Train Travel", "km", 0.04),
    ("LPG", "kg", 2.98),
    ("Flight", "km", 0.15),
]

# Simple rule-based suggestions for each category
SUGGESTIONS = {
    "Electricity": "Switch off unused lights/fans and prefer LED bulbs and efficient appliances.",
    "Car Travel": "Consider public transport, car-pooling or cycling for suitable journeys.",
    "Bus Travel": "Bus travel is already a good choice; walk or cycle for very short trips.",
    "Train Travel": "Train travel is a low-emission option; keep using it where possible.",
    "LPG": "Use lids while cooking and pressure cookers to save cooking gas.",
    "Flight": "Flights add a lot of CO2; prefer trains for shorter distances when possible.",
}

MONTH_NAMES = ("January", "February", "March", "April", "May", "June", "July",
               "August", "September", "October", "November", "December")

# Common SELECT used to show records with category name and unit (JOIN)
RECORD_QUERY = ("SELECT r.record_id, r.record_date, c.category_name, r.quantity, "
               "c.unit, r.co2_emission, r.note "
               "FROM emission_records r INNER JOIN categories c "
               "ON r.category_id = c.category_id ")

connection = None
cursor = None

# ----- Database setup -----

# Connect to the MySQL server and select (or create) the database
def connect_database():
    global connection, cursor
    connection = mysql.connector.connect(host=DB_HOST, user=DB_USER, password=DB_PASSWORD)
```

```

cursor = connection.cursor()
cursor.execute("CREATE DATABASE IF NOT EXISTS " + DB_NAME)
cursor.execute("USE " + DB_NAME)

# Create both tables if they do not exist yet
def create_tables():
    cursor.execute("""CREATE TABLE IF NOT EXISTS categories (
        category_id INT AUTO_INCREMENT PRIMARY KEY,
        category_name VARCHAR(30) NOT NULL UNIQUE,
        unit VARCHAR(10) NOT NULL,
        emission_factor FLOAT NOT NULL)""")
    cursor.execute("""CREATE TABLE IF NOT EXISTS emission_records (
        record_id INT AUTO_INCREMENT PRIMARY KEY,
        record_date DATE NOT NULL,
        category_id INT NOT NULL,
        quantity FLOAT NOT NULL,
        co2_emission FLOAT NOT NULL,
        note VARCHAR(100),
        FOREIGN KEY (category_id) REFERENCES categories(category_id))""")
    connection.commit()

# Insert the default categories only when the table is empty
def insert_default_categories():
    cursor.execute("SELECT COUNT(*) FROM categories")
    count = cursor.fetchone()[0]
    if count == 0:
        query = "INSERT INTO categories (category_name, unit, emission_factor) VALUES (%s, %s, %s)"
        for category in DEFAULT_CATEGORIES:
            cursor.execute(query, category)
        connection.commit()
        print("Default categories added.")

# ----- Input helper functions -----

# Wait for the user before showing the menu again
def pause_program():
    input("\nPress Enter to continue...")

def print_heading(title):
    print("\n" + "=" * 60)
    print(title.center(60))
    print("=" * 60)

# Keep asking until the user types a whole number
def get_integer(prompt):
    while True:
        try:
            return int(input(prompt))
        except ValueError:
            print("Invalid input! Please enter a whole number.")

# Keep asking until the user types a number greater than zero
def get_positive_number(prompt):
    while True:
        try:
            value = float(input(prompt))
            if value <= 0:

```

```

        print("Quantity must be greater than zero.")
    else:
        return value
except ValueError:
    print("Invalid input! Please enter a numeric value.")

# Return the date as 'YYYY-MM-DD' if it is valid, otherwise None
def validate_date(date_text):
    try:
        date_value = datetime.datetime.strptime(date_text, "%Y-%m-%d")
        return date_value.strftime("%Y-%m-%d")
    except ValueError:
        return None

# Ask for a date; an empty answer means today's date
def get_date(prompt):
    while True:
        date_text = input(prompt + " (YYYY-MM-DD, Enter for today): ").strip()
        if date_text == "":
            return str(datetime.date.today())
        valid_date = validate_date(date_text)
        if valid_date is None:
            print("Invalid date! Example of a correct date: 2026-08-15")
        else:
            return valid_date

def ask_yes_no(prompt):
    answer = input(prompt + " (Y/N): ").strip().upper()
    return answer == "Y"

# ----- Categories -----

# Display all categories in a table and return them as a list
def display_categories():
    cursor.execute("SELECT category_id, category_name, unit, emission_factor "
                   "FROM categories ORDER BY category_id")
    categories = cursor.fetchall()
    print("\n{<5}{<16}{<8}{>22}".format("ID", "CATEGORY", "UNIT", "FACTOR (kg CO2/unit)"))
    print("-" * 51)
    for category in categories:
        print("{<5}{<16}{<8}{>22.3f}".format(category[0], category[1],
                                              category[2], category[3]))

    return categories

# Fetch one category (id, name, unit, factor) from MySQL, or None
def get_category(category_id):
    cursor.execute("SELECT category_id, category_name, unit, emission_factor "
                   "FROM categories WHERE category_id = %s", (category_id,))
    return cursor.fetchone()

# Ask the user to pick a valid category ID
def choose_category():
    display_categories()
    while True:
        category_id = get_integer("\nEnter category ID: ")
        category = get_category(category_id)
        if category is None:

```

```

        print("No category with that ID. Try again.")
    else:
        return category

# ----- Carbon calculation -----

# Calculate carbon emission: CO2 = quantity x emission factor
def calculate_emission(quantity, emission_factor):
    return round(float(quantity) * float(emission_factor), 2)

# ----- Add and view records -----

def add_record():
    print_heading("ADD CARBON RECORD")
    category = choose_category()
    record_date = get_date("Enter date")
    quantity = get_positive_number("Enter quantity in " + category[2] + ": ")
    # Only the first 100 characters are kept because the note column is VARCHAR(100)
    note = input("Enter a note (optional): ").strip()[:100]

    # The emission factor comes from the categories table in MySQL
    co2 = calculate_emission(quantity, category[3])
    print("\nCalculation:", quantity, category[2], "x", category[3], "kg/" + category[2],
          "=", co2, "kg CO2")

    if ask_yes_no("Save this record?"):
        query = ("INSERT INTO emission_records (record_date, category_id, quantity, "
                  "co2_emission, note) VALUES (%s, %s, %s, %s, %s)")
        values = (record_date, category[0], quantity, co2, note)
        cursor.execute(query, values)
        connection.commit()
        print("Record saved successfully with ID", cursor.lastrowid)
    else:
        print("Record not saved.")

# Print a list of records as a neat table
def print_records(records):
    if len(records) == 0:
        print("\nNo records found.")
        return
    print("\n{<5}{<13}{<15}{>10} {<6}{>10} {}".format(
        "ID", "DATE", "CATEGORY", "QUANTITY", "UNIT", "CO2 (kg)", "NOTE"))
    print("-" * 80)
    for record in records:
        note = record[6] if record[6] else ""
        print("{<5}{<13}{<15}{>10.2f} {<6}{>10.2f} {}".format(
            record[0], str(record[1]), record[2], record[3], record[4], record[5], note))
    print("-" * 80)
    print("Records shown:", len(records))

# Display all stored records
def view_all_records():
    print_heading("ALL CARBON RECORDS")
    cursor.execute(RECORD_QUERY + "ORDER BY r.record_date, r.record_id")
    records = cursor.fetchall()
    print_records(records)

# Fetch one record by its ID, or None if it does not exist

```

```

def get_record(record_id):
    cursor.execute(RECORD_QUERY + "WHERE r.record_id = %s", (record_id,))
    return cursor.fetchone()

# ----- Search -----

def search_by_id():
    record_id = get_integer("Enter record ID: ")
    record = get_record(record_id)
    if record is None:
        print("No record found with ID", record_id)
    else:
        print_records([record])

def search_by_category():
    name = input("Enter category name (or part of it): ").strip()
    cursor.execute(RECORD_QUERY + "WHERE c.category_name LIKE %s ORDER BY r.record_date",
                  ("% " + name + "%",))
    print_records(cursor.fetchall())

def search_by_date():
    record_date = get_date("Enter date")
    cursor.execute(RECORD_QUERY + "WHERE r.record_date = %s", (record_date,))
    print_records(cursor.fetchall())

def search_by_date_range():
    start_date = get_date("Enter start date")
    end_date = get_date("Enter end date")
    if start_date > end_date:
        print("Start date was after end date, so the dates have been swapped.")
        start_date, end_date = end_date, start_date
    cursor.execute(RECORD_QUERY + "WHERE r.record_date BETWEEN %s AND %s "
                  "ORDER BY r.record_date", (start_date, end_date))
    print_records(cursor.fetchall())

def search_menu():
    while True:
        print_heading("SEARCH RECORDS")
        print("1. Search by Record ID")
        print("2. Search by Category")
        print("3. Search by Date")
        print("4. Search between two dates")
        print("5. Return to Main Menu")
        choice = input("\nEnter your choice: ").strip()
        if choice == "1":
            search_by_id()
        elif choice == "2":
            search_by_category()
        elif choice == "3":
            search_by_date()
        elif choice == "4":
            search_by_date_range()
        elif choice == "5":
            return
        else:
            print("Invalid choice! Please enter 1 to 5.")
            pause_program()

```

```
# ----- Update and delete -----

def update_record():
    print_heading("UPDATE RECORD")
    record_id = get_integer("Enter record ID to update: ")
    record = get_record(record_id)
    if record is None:
        print("No record found with ID", record_id)
        return
    print("\nCurrent details:")
    print_records([record])

    # The category ID is not part of RECORD_QUERY, so read it separately
    cursor.execute("SELECT category_id FROM emission_records WHERE record_id = %s",
                   (record_id,))
    category_id = cursor.fetchone()[0]
    quantity = record[3]
    new_date = str(record[1])
    new_note = record[6]

    print("\nPress Enter to keep the current value.")
    date_text = input("New date (YYYY-MM-DD): ").strip()
    if date_text != "":
        valid_date = validate_date(date_text)
        if valid_date is None:
            print("Invalid date, keeping the old date.")
        else:
            new_date = valid_date

    if ask_yes_no("Change the category?"):
        category_id = choose_category()[0]

    quantity_text = input("New quantity: ").strip()
    if quantity_text != "":
        try:
            if float(quantity_text) > 0:
                quantity = float(quantity_text)
            else:
                print("Quantity must be positive, keeping the old quantity.")
        except ValueError:
            print("Not a number, keeping the old quantity.")

    note_text = input("New note: ").strip()[:100]
    if note_text != "":
        new_note = note_text

    # Recalculate CO2 using the factor of the (possibly new) category
    category = get_category(category_id)
    new_co2 = calculate_emission(quantity, category[3])

    query = ("UPDATE emission_records SET record_date = %s, category_id = %s, "
            "quantity = %s, co2_emission = %s, note = %s WHERE record_id = %s")
    cursor.execute(query, (new_date, category_id, quantity, new_co2, new_note, record_id))
    connection.commit()
    print("\nRecord updated. Recalculated CO2:", new_co2, "kg")
    print_records([get_record(record_id)])

def delete_record():
    print_heading("DELETE RECORD")
    record_id = get_integer("Enter record ID to delete: ")
    record = get_record(record_id)
```

```

if record is None:
    print("No record found with ID", record_id)
    return
print_records([record])
if ask_yes_no("Delete this record?"):
    cursor.execute("DELETE FROM emission_records WHERE record_id = %s", (record_id,))
    connection.commit()
    print(cursor.rowcount, "record deleted.")
else:
    print("Delete cancelled.")

# ----- Summaries -----

# Return (count, total, average, minimum, maximum) using aggregate functions
def get_overall_totals():
    cursor.execute("SELECT COUNT(*), SUM(co2_emission), AVG(co2_emission), "
                   "MIN(co2_emission), MAX(co2_emission) FROM emission_records")
    return cursor.fetchone()

# Return a list of (category, number of records, total CO2), highest first
def get_category_totals():
    cursor.execute("SELECT c.category_name, COUNT(*), SUM(r.co2_emission) "
                   "FROM emission_records r INNER JOIN categories c "
                   "ON r.category_id = c.category_id "
                   "GROUP BY c.category_name ORDER BY SUM(r.co2_emission) DESC")
    return cursor.fetchall()

# Add up the total CO2 of all categories
def get_grand_total(category_totals):
    grand_total = 0
    for row in category_totals:
        grand_total = grand_total + float(row[2])
    return grand_total

def overall_summary():
    print_heading("CARBON FOOTPRINT SUMMARY")
    count, total, average, lowest, highest = get_overall_totals()
    if count == 0:
        print("No records available yet.")
        return
    print("Total records           :", count)
    print("Total CO2 emissions       : {:.2f} kg".format(total))
    print("Average CO2 per record      : {:.2f} kg".format(average))
    print("Lowest individual emission : {:.2f} kg".format(lowest))
    print("Highest individual emission: {:.2f} kg".format(highest))

    cursor.execute("SELECT MIN(record_date), MAX(record_date) FROM emission_records")
    first_date, last_date = cursor.fetchone()
    print("Records from", first_date, "to", last_date)

def category_summary():
    print_heading("CATEGORY-WISE SUMMARY")
    rows = get_category_totals()
    if len(rows) == 0:
        print("No records available yet.")
        return
    grand_total = get_grand_total(rows)
    print("{:<18}{:>10}{:>16}{:>12}".format("CATEGORY", "RECORDS", "TOTAL CO2", "SHARE"))

```

```

print("-" * 56)
for row in rows:
    share = float(row[2]) / grand_total * 100 if grand_total > 0 else 0
    print("{:<18}{:>10}{:>16.2f}{:>11.1f}%".format(row[0], row[1], row[2], share))
print("-" * 56)
print("{:<18}{:>10}{:>16.2f}".format("TOTAL", "", grand_total))

# Show which months have records, using DISTINCT
def show_available_months():
    cursor.execute("SELECT DISTINCT YEAR(record_date), MONTH(record_date) "
                  "FROM emission_records ORDER BY 1, 2")
    months = cursor.fetchall()
    if len(months) > 0:
        print("Months with records:")
        for year, month in months:
            print(" ", MONTH_NAMES[month - 1], year, "(" + str(month) + "/" + str(year) + ")")
    return len(months)

def monthly_summary():
    print_heading("MONTHLY SUMMARY")
    if show_available_months() == 0:
        print("No records available yet.")
        return
    month = get_integer("\nEnter month number (1-12): ")
    if month < 1 or month > 12:
        print("Invalid month! It must be between 1 and 12.")
        return
    year = get_integer("Enter year (e.g. 2026): ")

    cursor.execute("SELECT COUNT(*), SUM(co2_emission), AVG(co2_emission) FROM emission_records "
                  "WHERE MONTH(record_date) = %s AND YEAR(record_date) = %s", (month, year))
    count, total, average = cursor.fetchone()
    print("\nCARBON SUMMARY -", MONTH_NAMES[month - 1].upper(), year)
    if count == 0:
        print("No records for this month.")
        return
    print("\nTotal records:", count)
    print("Total estimated CO2: {:.2f} kg".format(total))
    print("Average emission: {:.2f} kg".format(average))

    # Category with the largest total in that month
    cursor.execute("SELECT c.category_name, SUM(r.co2_emission) "
                  "FROM emission_records r INNER JOIN categories c "
                  "ON r.category_id = c.category_id "
                  "WHERE MONTH(r.record_date) = %s AND YEAR(r.record_date) = %s "
                  "GROUP BY c.category_name ORDER BY SUM(r.co2_emission) DESC", (month, year))
    top = cursor.fetchall()[0]
    print("Highest category:", top[0], "{:.2f} kg".format(top[1]))

# Return a list of suggestion strings based on the recorded categories
def generate_suggestions(category_totals):
    suggestions = []
    grand_total = get_grand_total(category_totals)
    for row in category_totals:
        name = row[0]
        share = float(row[2]) / grand_total * 100 if grand_total > 0 else 0
        # Suggest only for categories contributing at least 20% of the total
        if share >= 20:
            if name in SUGGESTIONS:
                advice = SUGGESTIONS[name]

```

```

        else:
            advice = "Try to reduce this activity where it is practical."
            suggestions.append(name + " ({:.1f}% of total): ".format(share) + advice)
    return suggestions

def highest_emission_category():
    print_heading("HIGHEST EMISSION CATEGORY")
    rows = get_category_totals()
    if len(rows) == 0:
        print("No records available yet.")
        return
    # Rows are ordered by total CO2 in descending order, so the first is the highest
    print("Category with the highest emissions:", rows[0][0])
    print("Total CO2 from this category: {:.2f} kg in {} record(s)".format(rows[0][2], rows[0][1]))
    if len(rows) > 1:
        print("Lowest category:", rows[-1][0], "{:.2f} kg".format(rows[-1][2]))

    print("\nSuggestions:")
    for suggestion in generate_suggestions(rows):
        print("-", suggestion)

# ----- Export -----

# Write all records to a CSV file using the csv module
def export_to_csv():
    print_heading("EXPORT RECORDS TO CSV")
    cursor.execute(RECORD_QUERY + "ORDER BY r.record_date, r.record_id")
    records = cursor.fetchall()
    if len(records) == 0:
        print("No records to export.")
        return
    try:
        with open(CSV_FILE, "w", newline="", encoding="utf-8") as file:
            writer = csv.writer(file)
            writer.writerow(["Record ID", "Date", "Category", "Quantity", "Unit",
                             "CO2 (kg)", "Note"])
            writer.writerows(records)
        print(len(records), "records exported to", CSV_FILE)
    except IOError:
        print("Error: could not write to", CSV_FILE, "(is it open in another program?)")

# Write an overall and category-wise summary to a text file
def export_summary_to_text():
    print_heading("EXPORT SUMMARY TO TEXT FILE")
    count, total, average, lowest, highest = get_overall_totals()
    if count == 0:
        print("No records available to summarise.")
        return
    category_rows = get_category_totals()
    file = None
    try:
        file = open(SUMMARY_FILE, "w", encoding="utf-8")
        file.write("CARBON DATA LOGGER\n")
        file.write("PERSONAL FOOTPRINT SUMMARY\n")
        file.write("Generated on: " + str(datetime.date.today()) + "\n\n")
        file.write("Total Records: " + str(count) + "\n")
        file.write("Total Estimated CO2: {:.2f} kg\n".format(total))
        file.write("Average CO2: {:.2f} kg\n".format(average))
        file.write("Lowest Emission: {:.2f} kg\n".format(lowest))
        file.write("Highest Emission: {:.2f} kg\n\n".format(highest))

```

```

file.write("CATEGORY SUMMARY\n")
file.write("-----\n")
for row in category_rows:
    file.write("{}: {:.2f} kg ({} records)\n".format(row[0], row[2], row[1]))
file.write("\nSUGGESTIONS\n")
file.write("-----\n")
for suggestion in generate_suggestions(category_rows):
    file.write("- " + suggestion + "\n")
file.write("\nNote: emission factors are illustrative project values.\n")
print("Summary exported to", SUMMARY_FILE)
except IOError:
    print("Error: could not write to", SUMMARY_FILE)
finally:
    # The file is closed whether or not an error occurred
    if file is not None:
        file.close()

# ----- Manage emission factors -----

def update_emission_factor():
    category = choose_category()
    print("Current factor for", category[1], "is", category[3], "kg CO2 per", category[2])
    new_factor = get_positive_number("Enter new emission factor: ")
    cursor.execute("UPDATE categories SET emission_factor = %s WHERE category_id = %s",
                   (new_factor, category[0]))
    connection.commit()
    print("Emission factor updated. It will be used for new or edited records.")

def add_category():
    name = input("Enter new category name: ").strip()
    unit = input("Enter unit (e.g. kWh, km, kg): ").strip()
    if name == "" or unit == "":
        print("Name and unit cannot be empty.")
        return
    if len(name) > 30 or len(unit) > 10:
        print("Name can have at most 30 characters and unit at most 10.")
        return
    factor = get_positive_number("Enter emission factor (kg CO2 per " + unit + "): ")
    try:
        cursor.execute("INSERT INTO categories (category_name, unit, emission_factor) "
                       "VALUES (%s, %s, %s)", (name, unit, factor))
        connection.commit()
        print("Category", name, "added.")
    except mysql.connector.IntegrityError:
        print("A category with this name already exists.")

def manage_emission_factors():
    while True:
        print_heading("MANAGE / VIEW EMISSION FACTORS")
        print("Note: these factors are illustrative values for this project.")
        print("1. View Emission Factors")
        print("2. Update an Emission Factor")
        print("3. Add a New Category")
        print("4. Return to Main Menu")
        choice = input("\nEnter your choice: ").strip()
        if choice == "1":
            display_categories()
        elif choice == "2":
            update_emission_factor()
        elif choice == "3":

```

```

        add_category()
    elif choice == "4":
        return
    else:
        print("Invalid choice! Please enter 1 to 4.")
        pause_program()

# ----- Help and main menu -----

def show_help():
    print_heading("HELP")
    print("* Add a record for an activity, e.g. 15 kWh of electricity or 20 km by car.")
    print("* CO2 is estimated as: quantity x emission factor of the category.")
    print("* Dates are entered as YYYY-MM-DD. Press Enter to use today's date.")
    print("* Use Search, Update and Delete with the record ID shown in the tables.")
    print("* Summaries show totals overall, per category and per month.")
    print("* Exports create", CSV_FILE, "and", SUMMARY_FILE, "in this folder.")
    print("* Emission factors are illustrative and can be changed from option 12.")

def display_main_menu():
    print("\n" + "=" * 50)
    print("CARBON DATA LOGGER".center(50))
    print("=" * 50)
    print("1. Add Carbon Record")
    print("2. View All Records")
    print("3. Search Records")
    print("4. Update Record")
    print("5. Delete Record")
    print("6. Carbon Footprint Summary")
    print("7. Category-wise Summary")
    print("8. Monthly Summary")
    print("9. Highest Emission Category")
    print("10. Export Records to CSV")
    print("11. Export Summary to Text File")
    print("12. Manage/View Emission Factors")
    print("13. Help")
    print("0. Exit")

def main():
    try:
        # Connect to the MySQL database and prepare the tables
        connect_database()
        create_tables()
        insert_default_categories()
    except mysql.connector.Error as error:
        print("Could not connect to MySQL:", error)
        print("Check that MySQL is running and the password in DB_PASSWORD is correct.")
        return

    # Each menu number is linked to the function that handles it
    menu_actions = {
        "1": add_record, "2": view_all_records, "3": search_menu,
        "4": update_record, "5": delete_record, "6": overall_summary,
        "7": category_summary, "8": monthly_summary, "9": highest_emission_category,
        "10": export_to_csv, "11": export_summary_to_text,
        "12": manage_emission_factors, "13": show_help,
    }

    try:
        while True:

```

```

display_main_menu()
choice = input("\nEnter your choice: ").strip()
if choice == "0":
    print("Thank you for using Carbon Data Logger. Goodbye!")
    break
elif choice in menu_actions:
    try:
        menu_actions[choice]()
    except mysql.connector.Error as error:
        print("Database error:", error)
    # Sub-menus already pause after each action
    if choice not in ("3", "12"):
        pause_program()
else:
    print("Invalid choice! Please enter a number from 0 to 13.")
except KeyboardInterrupt:
    print("\nProgram stopped by the user.")
finally:
    # Always close the cursor and connection when the program ends
    cursor.close()
    connection.close()

```

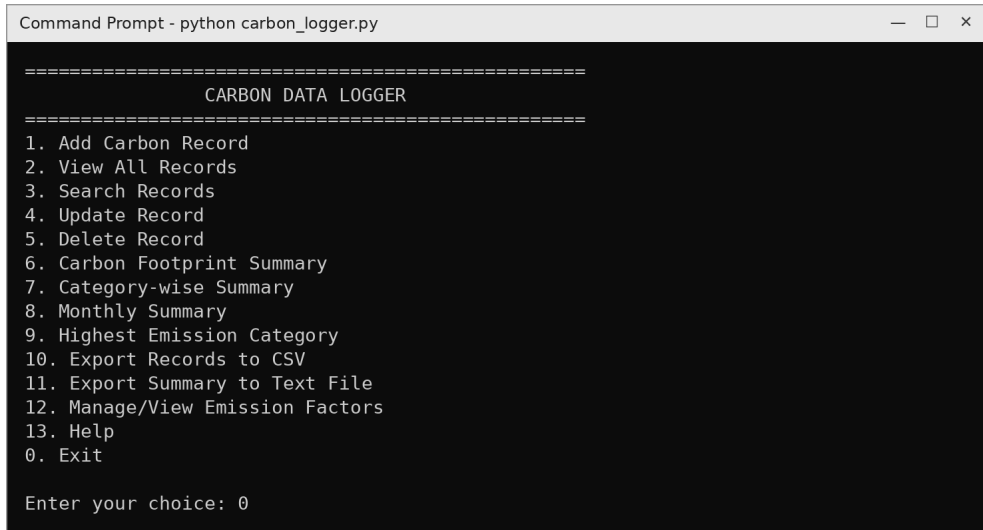
```

main()

```

9. Output Screenshots

The screenshots below were taken from an actual run of the program on the sample data.



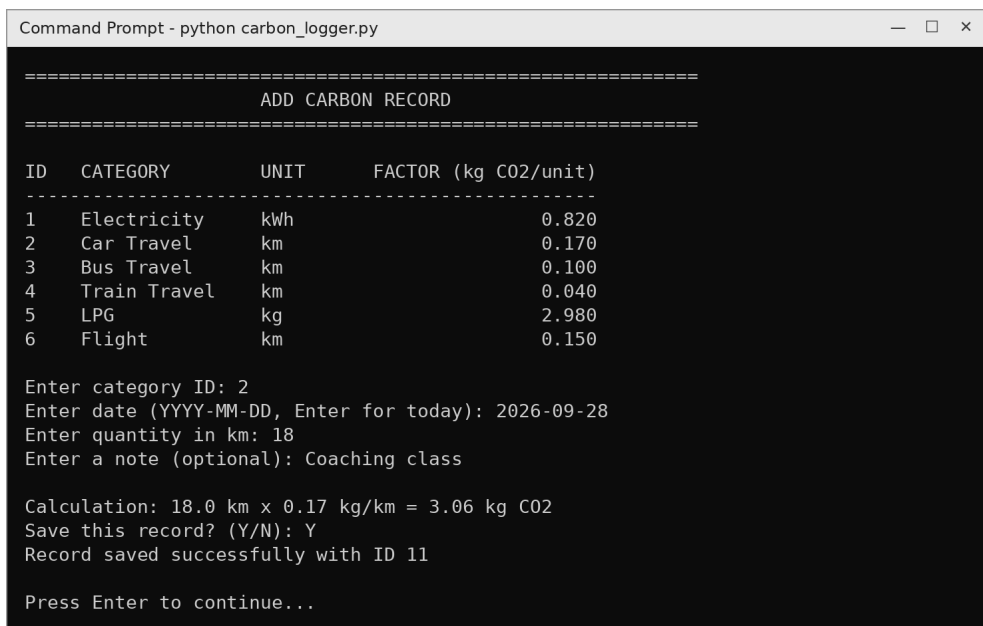
```

Command Prompt - python carbon_logger.py

=====
CARBON DATA LOGGER
=====
1. Add Carbon Record
2. View All Records
3. Search Records
4. Update Record
5. Delete Record
6. Carbon Footprint Summary
7. Category-wise Summary
8. Monthly Summary
9. Highest Emission Category
10. Export Records to CSV
11. Export Summary to Text File
12. Manage/View Emission Factors
13. Help
0. Exit

Enter your choice: 0
  
```

Figure 1: Main menu



```

Command Prompt - python carbon_logger.py

=====
ADD CARBON RECORD
=====

ID   CATEGORY      UNIT      FACTOR (kg CO2/unit)
-----
1    Electricity    kWh       0.820
2    Car Travel     km        0.170
3    Bus Travel     km        0.100
4    Train Travel   km        0.040
5    LPG            kg        2.980
6    Flight         km        0.150

Enter category ID: 2
Enter date (YYYY-MM-DD, Enter for today): 2026-09-28
Enter quantity in km: 18
Enter a note (optional): Coaching class

Calculation: 18.0 km x 0.17 kg/km = 3.06 kg CO2
Save this record? (Y/N): Y
Record saved successfully with ID 11

Press Enter to continue...
  
```

Figure 2: Adding a new record (option 1)

```

Command Prompt - python carbon_logger.py

=====
ALL CARBON RECORDS
=====

ID  DATE       CATEGORY      QUANTITY  UNIT   CO2 (kg)  NOTE
-----
1   2026-08-02  Electricity    12.50     kWh    10.25     Daily home use
2   2026-08-03  Car Travel     20.00     km      3.40      Trip to market
3   2026-08-05  Bus Travel     15.00     km      1.50      School bus
6   2026-08-12  Electricity    35.00     kWh    28.70     Monthly bill part
7   2026-08-20  Car Travel     42.00     km      7.14      Family outing
4   2026-09-01  LPG            2.00      kg      5.96      Cooking gas
8   2026-09-05  Train Travel   120.00    km      4.80      Trip to Jaipur
5   2026-09-10  Flight         800.00    km     120.00     Flight to Mumbai
9   2026-09-18  Electricity    28.00     kWh    22.96     Air conditioner use
10  2026-09-25  Bus Travel     16.00     km      1.60      School bus
11  2026-09-28  Car Travel     18.00     km      3.06      Coaching class

-----
Records shown: 11

Press Enter to continue...

```

Figure 3: Viewing all records (option 2)

```

Command Prompt - python carbon_logger.py

=====
SEARCH RECORDS
=====

1. Search by Record ID
2. Search by Category
3. Search by Date
4. Search between two dates
5. Return to Main Menu

Enter your choice: 4
Enter start date (YYYY-MM-DD, Enter for today): 2026-08-01
Enter end date (YYYY-MM-DD, Enter for today): 2026-08-31

ID  DATE       CATEGORY      QUANTITY  UNIT   CO2 (kg)  NOTE
-----
1   2026-08-02  Electricity    12.50     kWh    10.25     Daily home use
2   2026-08-03  Car Travel     20.00     km      3.40      Trip to market
3   2026-08-05  Bus Travel     15.00     km      1.50      School bus
6   2026-08-12  Electricity    35.00     kWh    28.70     Monthly bill part
7   2026-08-20  Car Travel     42.00     km      7.14      Family outing

-----
Records shown: 5

Press Enter to continue...

```

Figure 4: Searching between two dates using BETWEEN (option 3 → 4)

```

Command Prompt - python carbon_logger.py

=====
SEARCH RECORDS
=====

1. Search by Record ID
2. Search by Category
3. Search by Date
4. Search between two dates
5. Return to Main Menu

Enter your choice: 2
Enter category name (or part of it): elec

ID  DATE        CATEGORY        QUANTITY  UNIT   CO2 (kg)  NOTE
-----
1   2026-08-02   Electricity      12.50    kWh     10.25    Daily home use
6   2026-08-12   Electricity      35.00    kWh     28.70    Monthly bill part
9   2026-09-18   Electricity      28.00    kWh     22.96    Air conditioner use
-----
Records shown: 3

Press Enter to continue...

```

Figure 5: Searching by part of a category name using LIKE (option 3 → 2)

```

Command Prompt - python carbon_logger.py

=====
UPDATE RECORD
=====

Enter record ID to update: 2

Current details:

ID  DATE        CATEGORY        QUANTITY  UNIT   CO2 (kg)  NOTE
-----
2   2026-08-03   Car Travel      20.00    km       3.40    Trip to market
-----
Records shown: 1

Press Enter to keep the current value.
New date (YYYY-MM-DD):
Change the category? (Y/N): N
New quantity: 25
New note: Trip to market and back

Record updated. Recalculated CO2: 4.25 kg

ID  DATE        CATEGORY        QUANTITY  UNIT   CO2 (kg)  NOTE
-----
2   2026-08-03   Car Travel      25.00    km       4.25    Trip to market and back
-----
Records shown: 1

Press Enter to continue...

```

Figure 6: Updating a record; the CO2 value is recalculated (option 4)

```

Command Prompt - python carbon_logger.py

=====
DELETE RECORD
=====

Enter record ID to delete: 3

ID  DATE        CATEGORY        QUANTITY  UNIT   CO2 (kg)  NOTE
-----
3   2026-08-05   Bus Travel      15.00    km       1.50    School bus
-----
Records shown: 1
Delete this record? (Y/N): Y
1 record deleted.

Press Enter to continue...

```

Figure 7: Deleting a record after confirmation (option 5)

```

Command Prompt - python carbon_logger.py

=====
CARBON FOOTPRINT SUMMARY
=====
Total records           : 10
Total CO2 emissions     : 208.72 kg
Average CO2 per record  : 20.87 kg
Lowest individual emission : 1.60 kg
Highest individual emission: 120.00 kg
Records from 2026-08-02 to 2026-09-28

Press Enter to continue...

```

Figure 8: Overall carbon footprint summary (option 6)

```

Command Prompt - python carbon_logger.py

=====
CATEGORY-WISE SUMMARY
=====
CATEGORY      RECORDS      TOTAL CO2      SHARE
-----
Flight         1          120.00        57.5%
Electricity     3           61.91        29.7%
Car Travel      3           14.45         6.9%
LPG             1            5.96         2.9%
Train Travel    1            4.80         2.3%
Bus Travel      1            1.60         0.8%
-----
TOTAL                208.72

Press Enter to continue...

```

Figure 9: Category-wise summary with percentage share (option 7)

```

Command Prompt - python carbon_logger.py

=====
MONTHLY SUMMARY
=====
Months with records:
  August 2026 (8/2026)
  September 2026 (9/2026)

Enter month number (1-12): 9
Enter year (e.g. 2026): 2026

CARBON SUMMARY - SEPTEMBER 2026

Total records: 6
Total estimated CO2: 158.38 kg
Average emission: 26.40 kg
Highest category: Flight (120.00 kg)

Press Enter to continue...

```

Figure 10: Monthly summary for September 2026 (option 8)

```

Command Prompt - python carbon_logger.py

=====
HIGHEST EMISSION CATEGORY
=====
Category with the highest emissions: Flight
Total CO2 from this category: 120.00 kg in 1 record(s)
Lowest category: Bus Travel (1.60 kg)

Suggestions:
- Flight (57.5% of total): Flights add a lot of CO2; prefer trains for shorter distances when possible.
- Electricity (29.7% of total): Switch off unused lights/fans and prefer LED bulbs and efficient appliances.

Press Enter to continue...

```

Figure 11: Highest emission category and suggestions (option 9)

```

Command Prompt - python carbon_logger.py

=====
                        EXPORT RECORDS TO CSV
=====
10 records exported to carbon_records.csv

Press Enter to continue...

=====
                        EXPORT SUMMARY TO TEXT FILE
=====
Summary exported to carbon_summary.txt

Press Enter to continue...

```

Figure 12: Exporting records to CSV and summary to a text file (options 10 and 11)

```

Notepad - carbon_records.csv

Record ID,Date,Category,Quantity,Unit,C02 (kg),Note
1,2026-08-02,Electricity,12.5,kWh,10.25,Daily home use
2,2026-08-03,Car Travel,25.0,km,4.25,Trip to market and back
6,2026-08-12,Electricity,35.0,kWh,28.7,Monthly bill part
7,2026-08-20,Car Travel,42.0,km,7.14,Family outing
4,2026-09-01,LPG,2.0,kg,5.96,Cooking gas
8,2026-09-05,Train Travel,120.0,km,4.8,Trip to Jaipur
5,2026-09-10,Flight,800.0,km,120.0,Flight to Mumbai
9,2026-09-18,Electricity,28.0,kWh,22.96,Air conditioner use
10,2026-09-25,Bus Travel,16.0,km,1.6,School bus
11,2026-09-28,Car Travel,18.0,km,3.06,Coaching class

```

Figure 13: Contents of the exported carbon_records.csv

```

Notepad - carbon_summary.txt

CARBON DATA LOGGER
PERSONAL FOOTPRINT SUMMARY
Generated on: 2026-10-03

Total Records: 10
Total Estimated C02: 208.72 kg
Average C02: 20.87 kg
Lowest Emission: 1.60 kg
Highest Emission: 120.00 kg

CATEGORY SUMMARY
-----
Flight: 120.00 kg (1 records)
Electricity: 61.91 kg (3 records)
Car Travel: 14.45 kg (3 records)
LPG: 5.96 kg (1 records)
Train Travel: 4.80 kg (1 records)
Bus Travel: 1.60 kg (1 records)

SUGGESTIONS
-----
- Flight (57.5% of total): Flights add a lot of C02; prefer trains for shorter distances when possible.
- Electricity (29.7% of total): Switch off unused lights/fans and prefer LED bulbs and efficient appliances.

Note: emission factors are illustrative project values.

```

Figure 14: Contents of the exported carbon_summary.txt

```

Command Prompt - python carbon_logger.py

=====
MANAGE / VIEW EMISSION FACTORS
=====
Note: these factors are illustrative values for this project.
1. View Emission Factors
2. Update an Emission Factor
3. Add a New Category
4. Return to Main Menu

Enter your choice: 1

ID    CATEGORY      UNIT      FACTOR (kg CO2/unit)
-----
1     Electricity    kWh       0.820
2     Car Travel     km        0.170
3     Bus Travel     km        0.100
4     Train Travel   km        0.040
5     LPG            kg        2.980
6     Flight         km        0.150

Press Enter to continue...

```

Figure 15: Viewing the emission factors (option 12)

```

Command Prompt - python carbon_logger.py

=====
HELP
=====
* Add a record for an activity, e.g. 15 kWh of electricity or 20 km by car.
* CO2 is estimated as: quantity x emission factor of the category.
* Dates are entered as YYYY-MM-DD. Press Enter to use today's date.
* Use Search, Update and Delete with the record ID shown in the tables.
* Summaries show totals overall, per category and per month.
* Exports create carbon_records.csv and carbon_summary.txt in this folder.
* Emission factors are illustrative and can be changed from option 12.

Press Enter to continue...

```

Figure 16: Help screen (option 13)

```

Command Prompt - python carbon_logger.py

Enter your choice: 25
Invalid choice! Please enter a number from 0 to 13.

(main menu shown again)
Enter your choice: 1

=====
                        ADD CARBON RECORD
=====

ID   CATEGORY      UNIT      FACTOR (kg CO2/unit)
-----
1    Electricity    kWh       0.820
2    Car Travel     km        0.170
3    Bus Travel     km        0.100
4    Train Travel   km        0.040
5    LPG            kg        2.980
6    Flight         km        0.150

Enter category ID: 9
No category with that ID. Try again.

Enter category ID: 1
Enter date (YYYY-MM-DD, Enter for today): 2026-02-30
Invalid date! Example of a correct date: 2026-08-15
Enter date (YYYY-MM-DD, Enter for today): 2026-09-29
Enter quantity in kWh: abc
Invalid input! Please enter a numeric value.
Enter quantity in kWh: -4
Quantity must be greater than zero.
Enter quantity in kWh: 6
Enter a note (optional):

Calculation: 6.0 kWh x 0.82 kg/kWh = 4.92 kg CO2
Save this record? (Y/N): N
Record not saved.

Press Enter to continue...

(main menu shown again)
Enter your choice: 5

=====
                        DELETE RECORD
=====

Enter record ID to delete: 99
No record found with ID 99

Press Enter to continue...

```

Figure 17: Error handling: invalid menu choice, category ID, date, quantity and record ID

10. Testing

The program was tested with both valid and invalid inputs. All the test cases below gave the expected result.

#	Test	Input	Expected result
1	Invalid menu choice	25	"Invalid choice" message, menu shown again
2	Invalid category ID	9	"No category with that ID", asks again
3	Impossible date	2026-02-30	"Invalid date", asks again
4	Text as quantity	abc	"Please enter a numeric value", asks again
5	Negative quantity	-4	"Quantity must be greater than zero"
6	Correct record	Car Travel, 18 km	3.06 kg CO ₂ calculated and saved
7	Search date range	2026-08-01 to 2026-08-31	Only August records shown
8	Start date after end date	2026-08-31, 2026-08-01	Dates swapped, correct records shown
9	Update quantity	Record 2: 20 → 25 km	CO ₂ recalculated from 3.40 to 4.25 kg
10	Delete missing record	ID 99	"No record found with ID 99"
11	Cancel delete	N	"Delete cancelled", record kept
12	Duplicate category	"cng car" after "CNG Car"	"Category already exists"
13	Very long note	150 characters	First 100 characters saved
14	Note with ₹ / Hindi text	₹500 bill – बिजली	Saved and exported correctly
15	Wrong MySQL password	–	"Could not connect to MySQL" message
16	Summaries on empty database	Options 6–9	"No records available yet"
17	Ctrl+C at the menu	Ctrl+C	"Program stopped by the user", connection closed

11. Limitations and Future Scope

11.1 Limitations

- The emission factors are illustrative and not official values.
- The program is meant for a single user and has no login system.
- It works in the terminal only and has no graphical interface.
- Changing an emission factor does not recalculate old records; each record keeps the estimate made when it was saved.

11.2 Future scope

- Use official emission factors, for example from the Central Electricity Authority for electricity.
- Show monthly emissions as graphs.
- Let the user set a monthly CO₂ target and warn when it is crossed.
- Support several users, each with their own records.
- Build a graphical or mobile interface for easier use.

12. Bibliography

1. Computer Science – Textbook for Class XII, NCERT, New Delhi.
2. Computer Science Curriculum (Code 083), Class XII, Session 2026–27, Central Board of Secondary Education.
3. Sumita Arora, Computer Science with Python – Class XII, Dhanpat Rai & Co.
4. Python 3 Documentation – <https://docs.python.org/3/>
5. MySQL 8.0 Reference Manual – <https://dev.mysql.com/doc/refman/8.0/en/>
6. MySQL Connector/Python Developer Guide – <https://dev.mysql.com/doc/connector-python/en/>